

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score
```

```
df = pd.read_csv("/content/insurance.csv")
df
```

	age	sex	bmi	children	smoker	region	charges	
0	19	female	27.900	0	yes	southwest	16884.92400	
1	18	male	33.770	1	no	southeast	1725.55230	
2	28	male	33.000	3	no	southeast	4449.46200	
3	33	male	22.705	0	no	northwest	21984.47061	
4	32	male	28.880	0	no	northwest	3866.85520	
...	...	...	...	...	...	...	...	
1333	50	male	30.970	3	no	northwest	10600.54830	
1334	18	female	31.920	0	no	northeast	2205.98080	
1335	18	female	36.850	0	no	southeast	1629.83350	
1336	21	female	25.800	0	no	southwest	2007.94500	
1337	61	female	29.070	0	yes	northwest	29141.36030	

1338 rows × 7 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   charges     1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

```
df_encoded = pd.get_dummies(df, drop_first=True)
df_encoded.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest
0	19	27.900	0	16884.92400	False	True	False
1	18	33.770	1	1725.55230	True	False	False
2	28	33.000	3	4449.46200	True	False	False
3	33	22.705	0	21984.47061	True	False	True
4	32	28.880	0	3866.85520	True	False	True

Next steps:

[Generate code with df_encoded](#)[New interactive sheet](#)

```
X = df_encoded[['age', 'bmi', 'children', 'smoker_yes']]
y = df_encoded['charges']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)

y_pred_lr = lr.predict(X_test_scaled)

mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression MSE:", mse_lr)
print("Linear Regression R2:", r2_lr)
```

```
Linear Regression MSE: 33981653.95019775
Linear Regression R2: 0.7811147722517887
```

```
alphas = [0.1, 1, 10, 100]

ridge_results = {}

for alpha in alphas:
    ridge = Ridge(alpha=alpha)
    ridge.fit(X_train_scaled, y_train)

    y_pred = ridge.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    ridge_results[alpha] = (mse, r2)

print(f"\nAlpha = {alpha}")
print("MSE:", mse)
print("R2:", r2)
```

```
Alpha = 0.1
MSE: 33982227.34605835
R2: 0.7811110788505281
```

```
Alpha = 1
MSE: 33987477.22312754
R2: 0.781077262940931
```

```
Alpha = 10
MSE: 34048646.19152439
R2: 0.7806832567045574
```

```
Alpha = 100
MSE: 35377800.84957989
R2: 0.7721218040905247
```

```
print("\nComparison Table:")
print("Linear Regression -> MSE:", mse_lr, "R2:", r2_lr)

for alpha, values in ridge_results.items():
    print(f"Ridge (alpha={alpha}) -> MSE: {values[0]}, R2: {values[1]}")
```

```
Comparison Table:
Linear Regression -> MSE: 33981653.95019775 R2: 0.7811147722517887
Ridge (alpha=0.1) -> MSE: 33982227.34605835, R2: 0.7811110788505281
Ridge (alpha=1) -> MSE: 33987477.22312754, R2: 0.781077262940931
Ridge (alpha=10) -> MSE: 34048646.19152439, R2: 0.7806832567045574
Ridge (alpha=100) -> MSE: 35377800.84957989, R2: 0.7721218040905247
```

```
best_alpha = max(ridge_results, key=lambda x: ridge_results[x][1])
print("Best Alpha based on R2:", best_alpha)
```

```
Best Alpha based on R2: 0.1
```

```
alpha_values = []
r2_values = []

for alpha, values in ridge_results.items():
    alpha_values.append(alpha)
    r2_values.append(values[1])

plt.plot(alpha_values, r2_values)
plt.xlabel("Alpha")
plt.ylabel("R2 Score")
plt.title("Alpha vs R2 Score")
plt.show()
```

